

UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI
DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY



BACHELOR THESIS

By
Phung Dam Tien Si - 23BI14384

Title:
Design and Implementation of a Full-Stack
Web-Based Recruitment Platform for Students and
Employers

External Supervisor: Eng. Dinh Viet Ha

Internal Supervisor: Prof. Dr. Giang Anh Tuan

Hanoi, June - 2026

To whom it may concern,

I, (supervisor's name), certify that the thesis/ internship report of Mr/Ms. (student's name) is qualified to be presented in the Internship Jury 2025-2026.

Hanoi, <date>

Supervisor's signature

Acknowledgements

I would like to express my sincere gratitude to Mr. Dinh Viet Ha for his guidance, patience and mentorship throughout my internship. His support have greatly contributed to the completion of this project and my professional development.

I would also like to extend my sincere appreciation to SAVA META Technology Joint Stock Company for providing me with the opportunity to undertake my internship in a professional and supportive working environment.

Finally, I would like to thank everyone who has supported, encouraged, and accompanied me throughout my academic journey and personal development. Their encouragement has been a constant source of motivation during the completion of this thesis.

Contents

1	Introduction	1
1.1	Problem Statement	1
2	Literature Review	2
3	System Analysis	3
3.1	Proposed System	3
3.2	Functional Requirement	4
3.3	Non-Functional Requirement	4
3.4	Use Case Analysis	5
3.4.1	Shared Use Cases	5
3.4.2	Specialized Use Cases	7
3.5	Use Case Specification	8
3.5.1	Job Seeker: Apply for Job	9
3.5.2	Employer: Publish Job Posting	11
3.5.3	Employer: Review Application	13
4	System Design	15
4.1	Design Overview	15
4.2	System Architecture	16
4.2.1	System Context	16
4.2.2	Container View	16
4.2.3	Component View: Frontend	18
4.2.4	Component View: Backend	19
4.3	Database Design	21
4.3.1	Entity Relationship Overview	21
4.3.2	Company and Job Posting	22
4.3.3	Job Application	22
4.4	Backend Design	23
4.4.1	Backend Project Structure	24
4.4.2	Shared Backend Components	25
4.4.3	Job Application Module	25
4.4.4	Employer Job Management Module	26
4.5	Frontend Design	28
4.5.1	Job Application UI	28
4.5.2	Employer Job Management UI	28
4.6	Summary	29

5	Implementation	30
5.1	Overview	30
5.1.1	Request Pipeline	30
5.1.2	Error Handling	31
5.2	Backend Implementation	31
5.2.1	Job Application Module	31
5.2.2	Employer Job Management Module	32
5.2.3	Remaining Modules	34
5.3	Frontend Implementation	35
5.3.1	Job Application UI	35
5.3.2	Employer Job Management UI	35
5.3.3	Remaining Pages	36
5.4	Summary	36
6	Testing	37
7	Conclusion	38

List of Figures

3.1	Guest Use Case Diagram.	6
3.2	Authenticated User Use Case Diagram.	6
3.3	Job Seeker Use Case Diagram.	7
3.4	Employer Use Case Diagram.	8
4.1	System Context Diagram.	16
4.2	Container Diagram.	17
4.3	Frontend Component Diagram.	18
4.4	Backend Component Diagram.	20
4.5	Entity Relationship Diagram.	21
4.6	Company and Job Posting Entity Relationship Diagram.	22
4.7	Job Application Entity Relationship Diagram.	23
4.8	Job Application Module Class Diagram.	26
4.9	Company Profile Management Class Diagram.	27
4.10	Job Posting Management Class Diagram.	27
4.11	Employer Application Management Class Diagram.	27

List of Tables

3.1	Summary of Functional Requirements.	4
3.2	Summary of Non-Functional Requirements.	5
3.3	Use Case Specification: Apply for Job.	9
3.4	Use Case Specification: Publish Job Posting.	11
3.5	Use Case Specification: Review Application.	13
4.1	Design Principles and Rationale.	15
4.2	Backend Project Structure.	24
5.1	HTTP Status Code Conventions.	31
5.2	Job Application Endpoints.	31
5.3	Job Posting Endpoints.	33
5.4	Employer Application Review Endpoints.	34

Abbreviations

API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, and Delete
CV	Curriculum Vitae
DTO	Data Transfer Object
ERD	Entity Relationship Diagram
FK	Foreign Key
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model-View-Controller
ORM	Object Relational Mapping
PK	Primary Key
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDLC	Software Development Life Cycle
SPA	Single-Page Application
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
UX	User Experience
WDLC	Web Development Life Cycle

Chapter 1

Introduction

1.1 Problem Statement

Hiring used to run on paper. Job seekers printed resumes and delivered them in person or by post, and employers advertised openings in newspapers and reviewed applications by hand. Online recruitment platforms have changed this: employers now publish vacancies and candidates apply electronically, so distance and response time no longer decide how fast a role is filled.

The new approach brings its own friction. Each recruitment site has its own search interface, its own resume format, and its own application flow. A candidate who wants to reach more than one pool of jobs must recreate essentially the same profile and resume on every platform, then visit each one separately to check whether an application has progressed. The more platforms a job seeker uses, the more duplicated effort the search costs.

Employers face a similar mess in reverse. Publishing a posting is only the first step; they still have to collect applications, read resumes, record decisions, and keep those records straight. Without a single place that ties postings, applications, and decisions together, the employer ends up stitching information across tools — recreating the inefficiency the platform was supposed to remove.

These problems motivate a centralized system: one platform where job seekers search jobs, maintain a profile, upload a resume, apply, and track their applications, and where employers manage company information, publish postings, and review applicants through one interface. Because such a system stores personal data and lets users modify shared records, access must be controlled. Registration and login protect each user's data, and role-based permissions decide what each user may do: job seekers manage their profiles and applications, employers manage postings and candidates, and a back-office role keeps platform content in check. This thesis designs and implements that system, focusing on two central flows — applying for jobs on the job seeker's side, and managing postings and their applicants on the employer's side.

Chapter 2

Literature Review

Chapter 3

System Analysis

3.1 Proposed System

The challenges described in Chapter 1 motivate a centralized online recruitment platform. In this system, a job seeker can search for jobs, keep a personal profile, upload resumes, submit applications, and follow their progress. An employer can register a company, publish job postings, and review applicants. A back-office role moderates content and manages accounts. Putting these operations in one system replaces the fragmented workflow of the current market with a single consistent interface.

The system’s functionality is split into clear modules by business responsibility:

- **Job Discovery** allows guests to search active job postings and view company profiles.
- **Authentication** covers registration, login, logout, and password recovery.
- **Profile Management** lets users maintain personal information, job preferences, avatar, and account password.
- **Resume Management** lets job seekers upload and delete resumes used in applications.
- **Job Application** lets a job seeker apply to an active posting, track submitted applications, and withdraw them while still eligible.
- **Employer Job Management** lets an employer manage a company profile, run the full lifecycle of a job posting, and review the applications a posting receives.
- **Administration** supports moderating job postings and managing users and logs.

Because the platform stores personal data and lets users modify shared records, access is controlled. Registration and login protect each user’s account, and role-based permissions decide what each user may do. A job seeker manages their own profile and applications; an employer manages only their own company, postings, and candidates; a back-office worker moderates platform content.

This thesis develops the platform in full but examines two flows in depth: the **Job Application** flow on the job seeker’s side and the **Employer Job Management** flow on the employer’s side. These two flows form the round trip of a recruitment event — a job seeker submits an application, and an employer reviews it — and are therefore used to illustrate the analysis, design, and implementation in the remaining chapters.

3.2 Functional Requirement

Functional requirements define what the platform must do for its stakeholders. They are organized into modules based on business responsibility, as summarized in Table 3.1.

Table 3.1: Summary of Functional Requirements.

Module	Description
Authentication	Supports user registration, login, logout, and password recovery to ensure secure access to the system.
User Profile Management	Allows job seekers to maintain personal information, update job preferences, and manage uploaded resumes.
Job Discovery	Provides browsing, searching, filtering, and viewing of available job postings.
Job Application	Enables job seekers to submit applications, withdraw applications before processing, and monitor application status.
Company Management	Allows employers to create and maintain company information presented to prospective applicants.
Job Posting Management	Allows employers to create, edit, publish, close, and manage job postings throughout their recruitment life-cycle.
Recruitment Management	Supports employers in reviewing submitted applications and updating the recruitment status of candidates.
Administration	Allows administrators to manage user accounts, moderate job postings, and maintain overall platform integrity.

3.3 Non-Functional Requirement

Non-functional requirements describe the quality attributes the platform must satisfy, such as security, performance, usability, reliability, maintainability, and compatibility. Table 3.2 summarizes the main ones considered during design.

Table 3.2: Summary of Non-Functional Requirements.

Category	Description
Security	The system shall authenticate users before granting access to protected resources and enforce role-based authorization. Sensitive credentials shall be securely stored using password hashing techniques.
Performance	The system shall provide responsive access to common operations such as browsing job postings, submitting applications, and managing recruitment information.
Usability	The system shall provide a responsive web interface with consistent navigation, clear validation messages, and an intuitive user experience for all supported roles.
Reliability	The system shall ensure data consistency through proper validation, controlled data modification, and appropriate error handling mechanisms.
Maintainability	The system shall adopt a modular architecture with clearly separated components to simplify future maintenance and feature extension.
Compatibility	The system shall support modern desktop web browsers without requiring additional client-side software installation.

3.4 Use Case Analysis

Use case analysis models how users interact with the platform and shows the functional scope from each user role’s perspective. The platform serves three primary roles: Job Seeker, Employer, and Administrator. Each role performs different functions and uses a different subset of the system.

3.4.1 Shared Use Cases

To keep the model compact, the actors are organized with UML generalization. Signed-in users are represented by the generalized **Authenticated User** actor, and visitors are represented by the **Guest** actor.

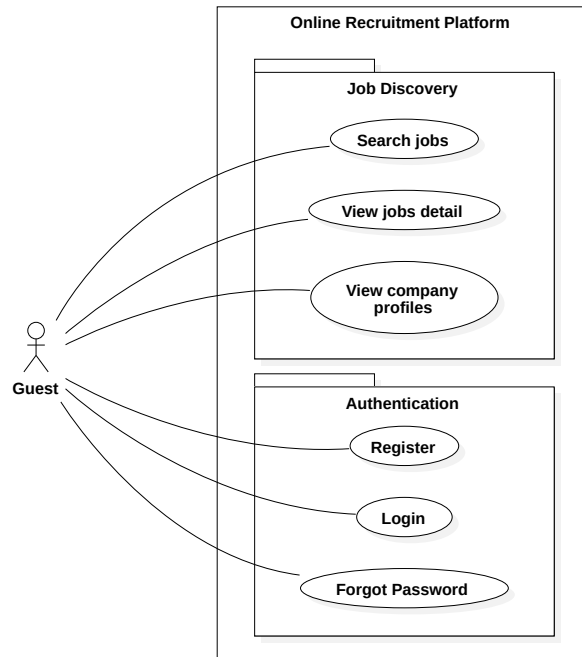


Figure 3.1: Guest Use Case Diagram.

As shown in Figure 3.1, guests can browse job postings, view company information, and register. These public functions give visitors access to the platform without requiring an account.

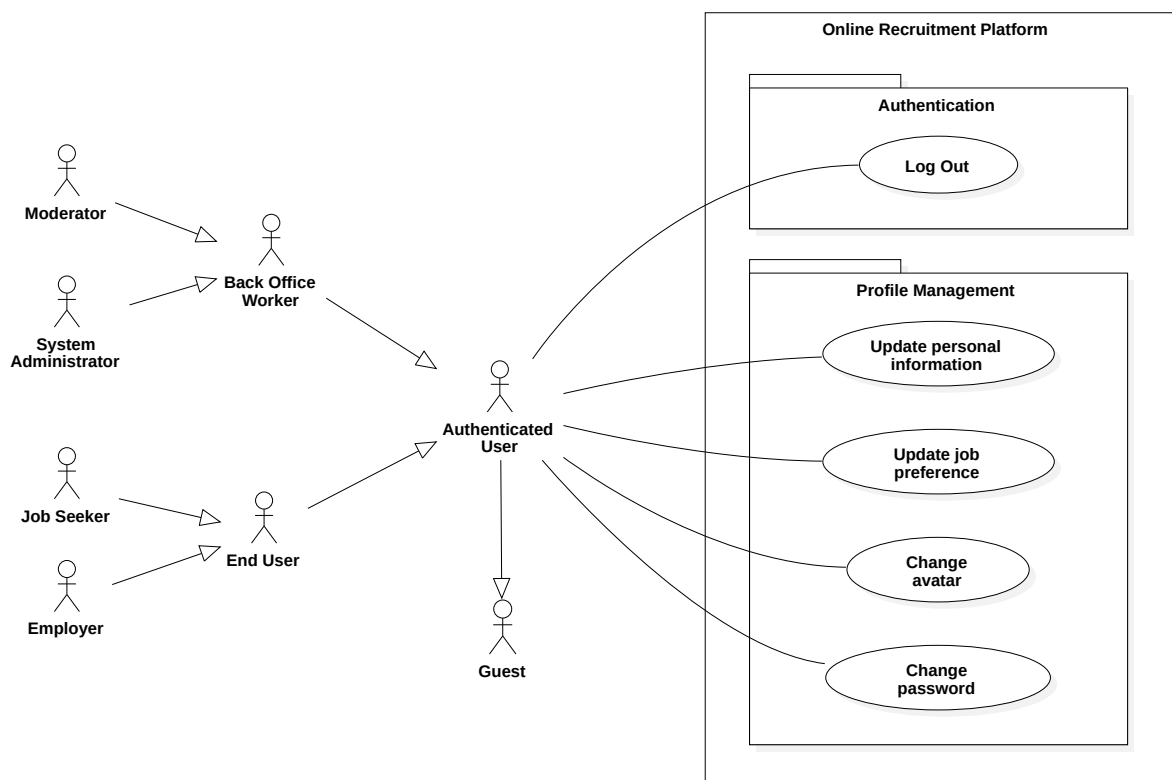


Figure 3.2: Authenticated User Use Case Diagram.

Figure 3.2 covers the functions common to all signed-in users: managing account information, changing the password, and logging out. Business-specific functions are

introduced through the specialized actors in the next section.

3.4.2 Specialized Use Cases

The following subsections present the role-specific use cases. In line with the scope defined in Section 3.1, the Job Seeker and Employer roles are examined in detail as the two featured modules of this thesis; the Back Office Worker use cases are presented in Appendix ??.

Job Seeker Use Cases

The Job Seeker is the main user on the demand side of the platform: this actor looks for jobs, keeps a profile, and takes part in recruitment through job applications.

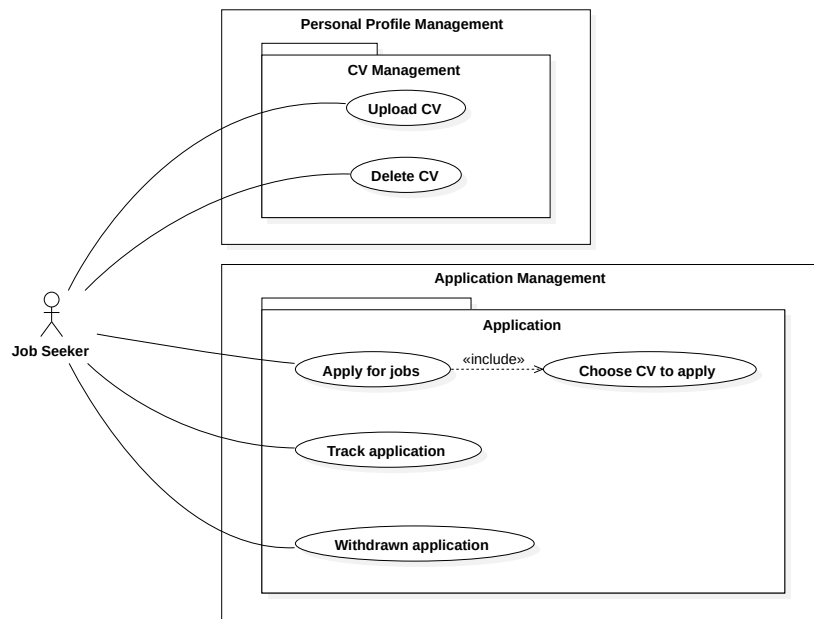


Figure 3.3: Job Seeker Use Case Diagram.

Figure 3.3 shows the Job Seeker's capabilities. Personal profile maintenance is part of the authenticated account functions; the recruitment-specific behavior of this actor is the **Job Application** flow: applying to an active posting with an uploaded resume, tracking submitted applications, and withdrawing an application while it is still in its initial state.

Employer Use Cases

The Employer represents the organizations and recruiters who publish job postings and evaluate applicants.

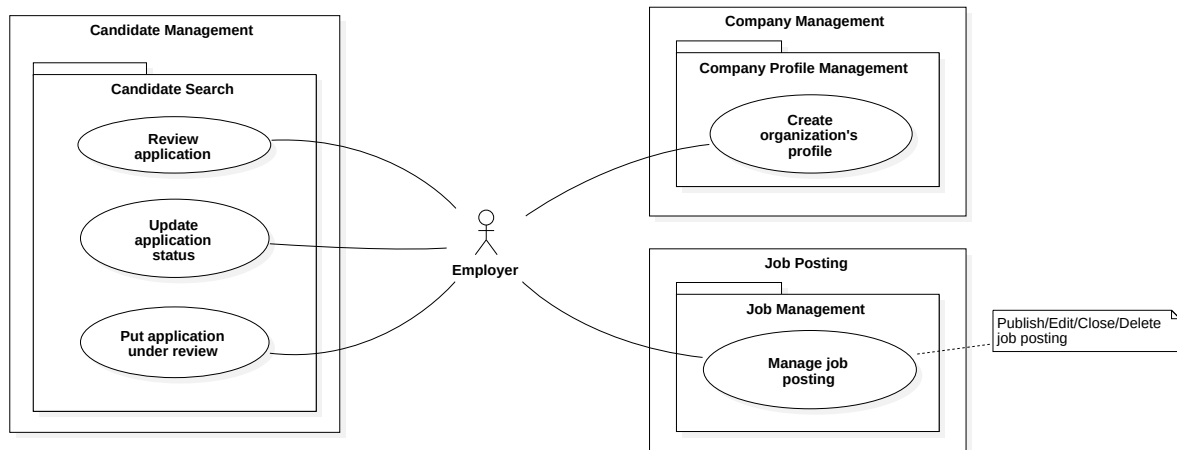


Figure 3.4: Employer Use Case Diagram.

Figure 3.4 shows the **Employer Job Management** module, which covers the complete recruitment lifecycle on the employer's side: creating a company profile, publishing and managing job postings (edit, close, re-open, delete), listing applications received for a posting, and moving each application through review until it is accepted or rejected.

3.5 Use Case Specification

The use case diagrams above give an overview of the system's functional scope. This section specifies the main use cases of the two featured modules in detail. Specifications for the remaining modules are provided in the appendix.

3.5.1 Job Seeker: Apply for Job

Table 3.3: Use Case Specification: Apply for Job.

Field	Specification
Use Case ID	UC-JS-01
Use Case Name	Apply for Job
Primary Actor	Job Seeker
Goal	Submit a job application to an employer.
Description	Allows a Job Seeker to submit an application for a published job posting.
Preconditions	• Job Seeker is authenticated. • The job posting is open for applications. • Job Seeker has an uploaded resume.
Trigger	The Job Seeker selects the Apply option on a job posting.
Main Success Scenario	1. Job Seeker views the job posting. 2. Job Seeker selects Apply. 3. System displays the application form and the job summary. 4. Job Seeker selects a resume. 5. Job Seeker submits the application. 6. System validates that the job is active, the resume belongs to the user, and no duplicate application exists. 7. System records the application as SUBMITTED. 8. System confirms the submission.

Table 3.3 (Continued)

Field	Specification
Alternative Flows	A1. No resume uploaded 1. System asks the Job Seeker to upload a resume first. A2. Already applied 1. System rejects the submission. 2. System informs the Job Seeker that an application already exists. A3. Posting no longer active 1. System rejects the submission. 2. System informs the Job Seeker that the posting is no longer available.
Postconditions	• A new application with status SUBMITTED is created.

3.5.2 Employer: Publish Job Posting

Table 3.4: Use Case Specification: Publish Job Posting.

Field	Specification
Use Case ID	UC-EMP-01
Use Case Name	Publish Job Posting
Primary Actor	Employer
Goal	Publish a job posting for potential applicants.
Description	Allows an Employer to create and publish a new job posting.
Preconditions	• Employer is authenticated. • Employer has a registered company profile.
Trigger	Employer selects the Post a Job function.
Main Success Scenario	1. Employer fills in the job title, description, requirements, location, employment type, and deadline. 2. Employer submits the posting. 3. System validates the input and checks that the deadline is in the future. 4. System assigns the employer's company to the posting. 5. System creates the posting with status <code>PENDING_APPROVAL</code>. 6. System confirms the creation and shows the posting in the employer's list.
Alternative Flows	A1. No company profile 1. System requires the Employer to create a company profile first. A2. Invalid input 1. System identifies the invalid fields. 2. Employer corrects and resubmits.

Table 3.4 (Continued)

Field	Specification
Postconditions	• A job posting with status <code>PENDING_APPROVAL</code> is stored. • The posting is hidden from public search until approved.

3.5.3 Employer: Review Application

Table 3.5: Use Case Specification: Review Application.

Field	Specification
Use Case ID	UC-EMP-06, UC-EMP-07, UC-EMP-08
Use Case Name	Review Application
Primary Actor	Employer
Goal	Review the applications a posting received and update their status.
Description	Allows an Employer to view the applications submitted to a posting and move each one through the review pipeline.
Preconditions	• Employer is authenticated. • The posting belongs to the employer's company.
Trigger	Employer opens the application list of a job posting.
Main Success Scenario	1. Employer selects a job posting. 2. System lists the applications with candidate name, resume, and status. 3. Employer opens an application's detail. 4. Employer marks the application <code>UNDER_REVIEW</code>. 5. Employer decides to accept or reject the application, with an optional reason on rejection. 6. System updates the application to <code>ACCEPTED</code> or <code>REJECTED</code> and records the decision time.
Alternative Flows	A1. Application withdrawn 1. System blocks further processing of the application. A2. Application already decided 1. System rejects the status change. A3. Application not under review 1. System only allows accept or reject on applications that are <code>UNDER_REVIEW</code>.

Table 3.5 (Continued)

Field	Specification
Postconditions	• The application is ACCEPTED or REJECTED, with the decision time recorded.

Chapter 4

System Design

4.1 Design Overview

This chapter presents the technical design of the proposed system based on the requirements identified in Chapter 3. It describes the system architecture, database design, backend structure, and frontend organization that form the blueprint for the implementation discussed in Chapter 5.

The design follows a small set of guiding principles, each addressing one or more non-functional requirements identified previously. Table 4.1 summarizes these principles and their rationale.

Table 4.1: Design Principles and Rationale.

Principle	Rationale
Layered Architecture	Separates presentation, API, business logic, and data access concerns, improving maintainability and testability.
Feature-Oriented Modular Design	Organizes code by business domain (e.g., Job, Application) rather than by technical type, keeping related logic together and easing future extension.
Thin Controller, Rich Service	Controllers handle only request/response concerns; business logic and validation reside in services, reducing duplication and improving reusability.
Security by Design	Authentication and authorization are consistently enforced to protect restricted resources, while credentials are securely stored using industry-standard cryptographic techniques.
Data Integrity	Relational constraints, foreign keys, and database integrity rules ensure data consistency and prevent invalid relationships.

These principles are applied consistently across all modules of the system. They are illustrated in detail for the two featured modules, Job Application and Employer Job Management; the design of the remaining modules is documented in the appendix.

The remainder of this chapter is organized as follows. Section 4.2 presents the system architecture, including the C4-based container view. Section 4.3 details the

database design and entity-relationship model for the featured modules. Section 4.4 describes the backend design, including the layered structure and module organization. Section 4.5 covers the frontend design. Section 4.6 summarizes the chapter.

4.2 System Architecture

The system architecture is documented using the C4 model, which describes a software system at progressively narrower levels of zoom: Context, Container, and Component. This progression matches the layered, modular design principles established in Section 4.1, letting each stakeholder read only the level of detail relevant to them.

4.2.1 System Context

The System Context diagram identifies the system's boundary and its interactions with external actors, without describing internal structure.

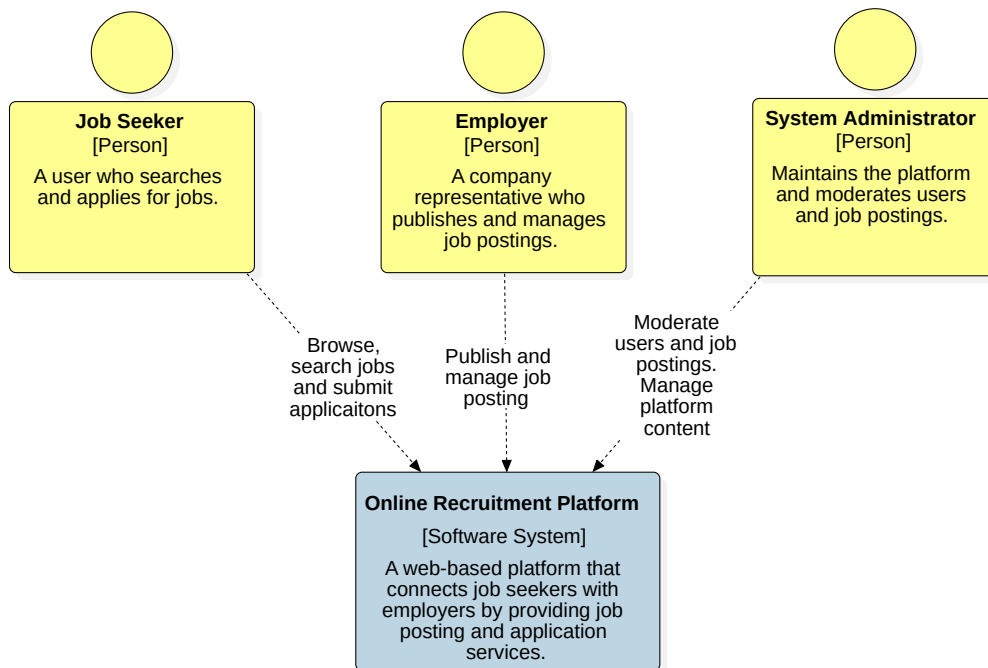


Figure 4.1: System Context Diagram.

As shown in Figure 4.1, three actors interact with the platform: the Job Seeker browses and applies for jobs, the Employer publishes and manages job postings, and the System Administrator moderates users and platform content. These actors correspond to the roles established in Chapter 3.

4.2.2 Container View

The Container diagram decomposes the system into its major deployable units and shows how they communicate.

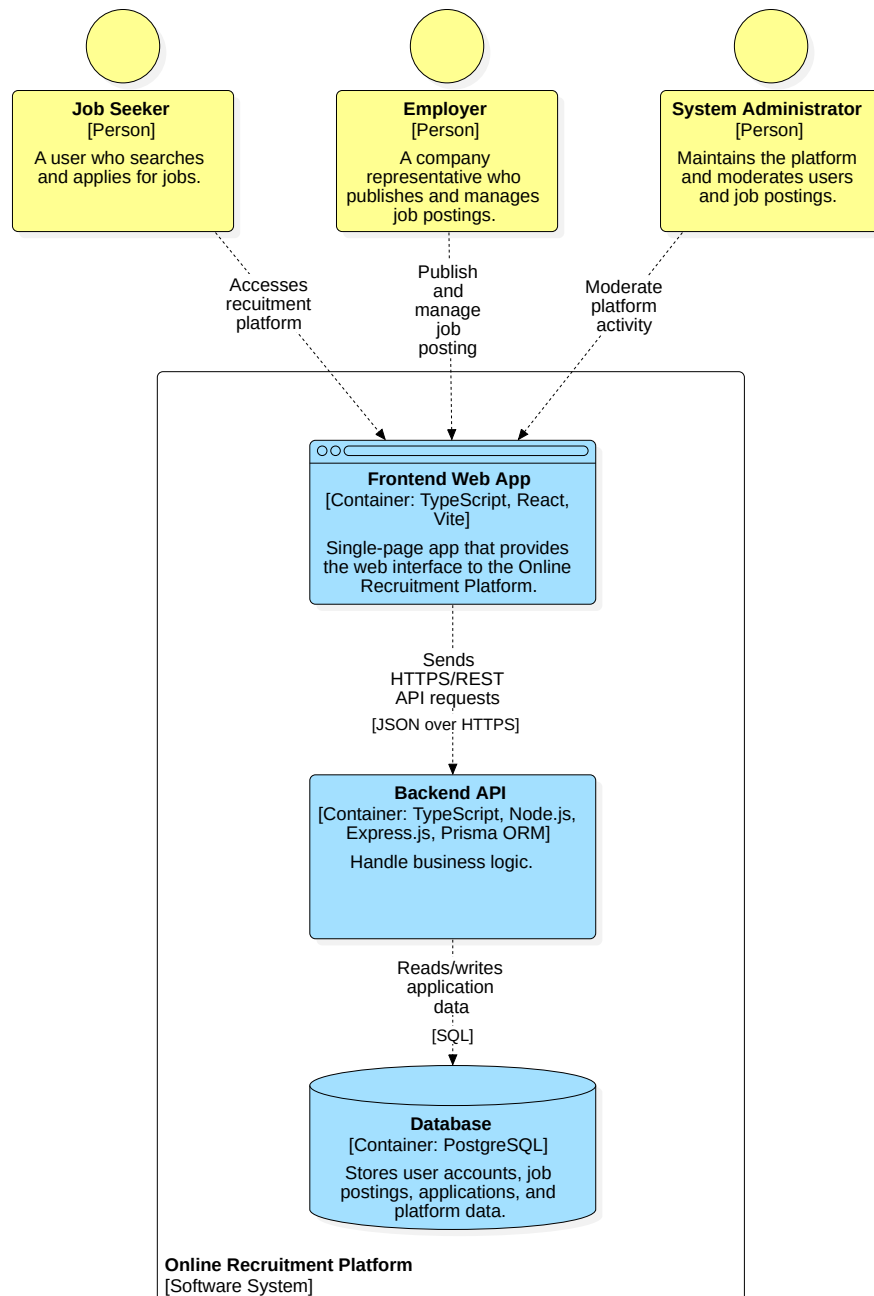


Figure 4.2: Container Diagram.

As illustrated in Figure 4.2, the system consists of three containers:

- The **Frontend Web App** is a React single-page application that renders the user interface.
- The **Backend API** is a Node.js/Express service, built with TypeScript and Prisma ORM, that implements all business logic and exposes it as a REST API.
- The **Database** is a PostgreSQL instance that persists user accounts, job postings, applications, and other platform data.

The Frontend communicates with the Backend over HTTPS using JSON, and the Backend communicates with the Database using SQL through Prisma.

4.2.3 Component View: Frontend

The Frontend component diagram decomposes the Frontend Web App container into its internal building blocks.

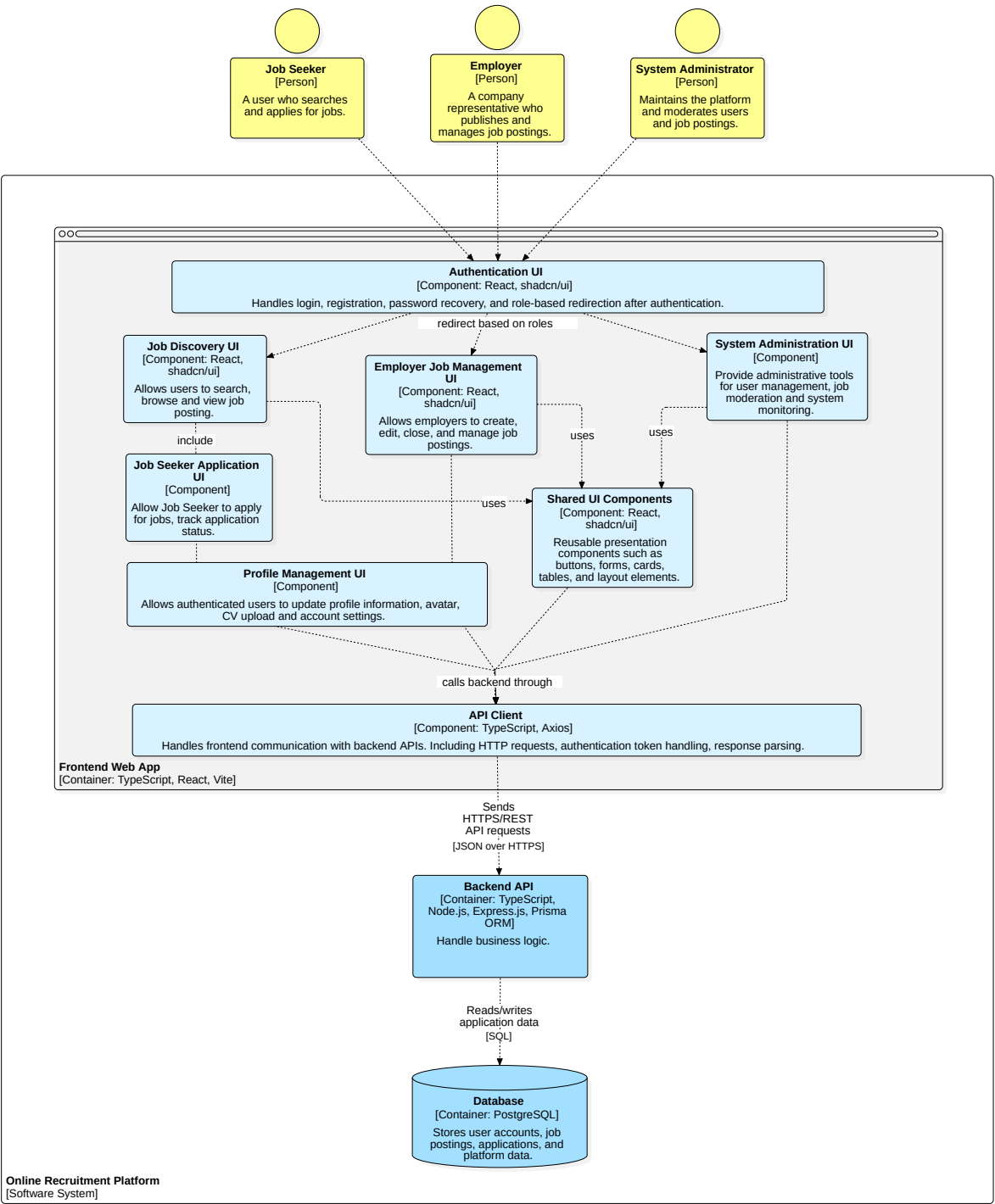


Figure 4.3: Frontend Component Diagram.

As shown in Figure 4.3, the frontend is organized around role-oriented feature components:

- The **Job Seeker Application UI** supports job application submission and status tracking.

- The **Profile Management UI** manages personal information, avatar, and resumes.
- The **Employer Job Management UI** supports company and job posting management.
- The **Authentication UI** handles login, registration, and role-based redirection.
- The **Job Discovery UI** supports job browsing and company profile viewing.

All feature components reuse a common Shared UI Components library and communicate with the backend exclusively through a centralized API Client, which handles HTTP requests, authentication tokens, and response parsing.

4.2.4 Component View: Backend

The Backend component diagram decomposes the Backend API container into functional components that implement the platform's business capabilities.

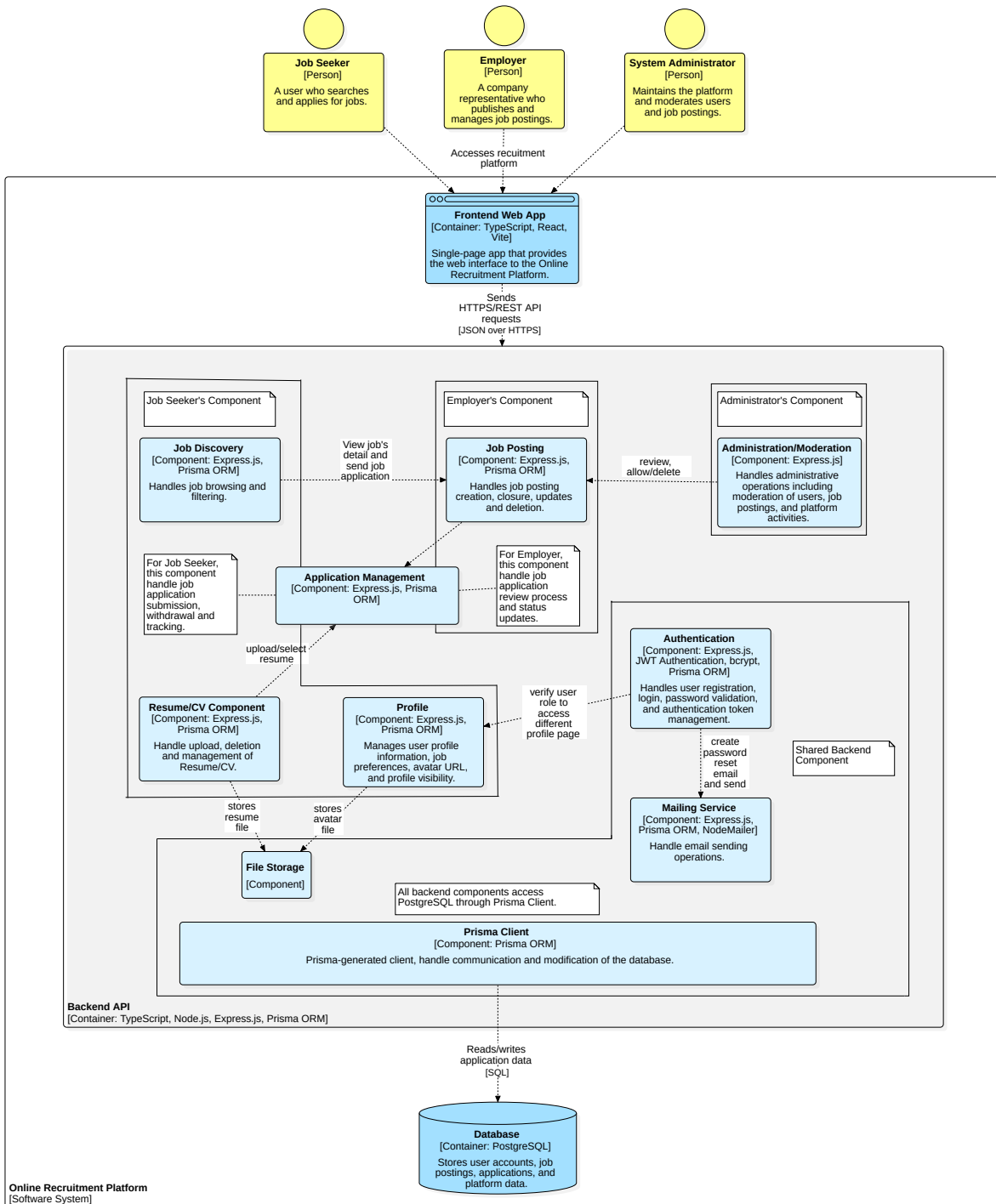


Figure 4.4: Backend Component Diagram.

As illustrated in Figure 4.4, the backend is organized into role-oriented business components:

- The **Application Management** components support the job seeker's workflow of applying and tracking.
- The **Job Posting** and **Employer Application Management** components provide employers with job publishing and application review capabilities.

- The **Administration/Moderation** component enables platform administrators to manage users and job postings.

Common functionality is centralized in shared infrastructure components. The Authentication component manages user registration, login, and authorization using JWT-based authentication, while the Mailing Service handles email notifications. File assets are stored through the File Storage component.

All business components interact with the PostgreSQL database through the shared **Prisma Client**, ensuring a consistent data access mechanism across the application.

4.3 Database Design

The database design defines how the system’s persistent data is organized and managed. It identifies the main business entities, their relationships, and the constraints that ensure data consistency. The design follows the principles introduced in Section 4.1, particularly data integrity, by using a normalized relational schema with clearly defined primary and foreign key relationships.

4.3.1 Entity Relationship Overview

Figure 4.5 presents an overview of the complete database schema used by the platform. To improve readability, the overview diagram includes the primary entities, key attributes, and relationships while omitting system-generated fields such as timestamps and audit metadata.

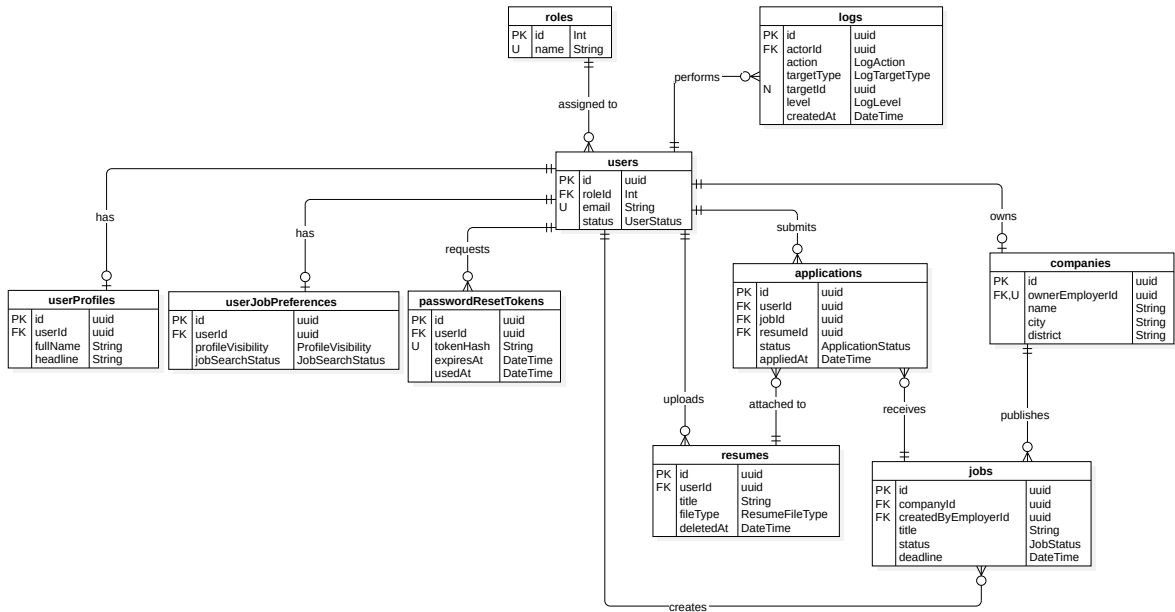


Figure 4.5: Entity Relationship Diagram.

The remainder of this section details the entity groups that back the two featured modules — the company and job posting domain and the job application domain. The entities supporting the remaining modules are described in the appendix.

4.3.2 Company and Job Posting

Figure 4.6 details the entities involved in company and job posting management.

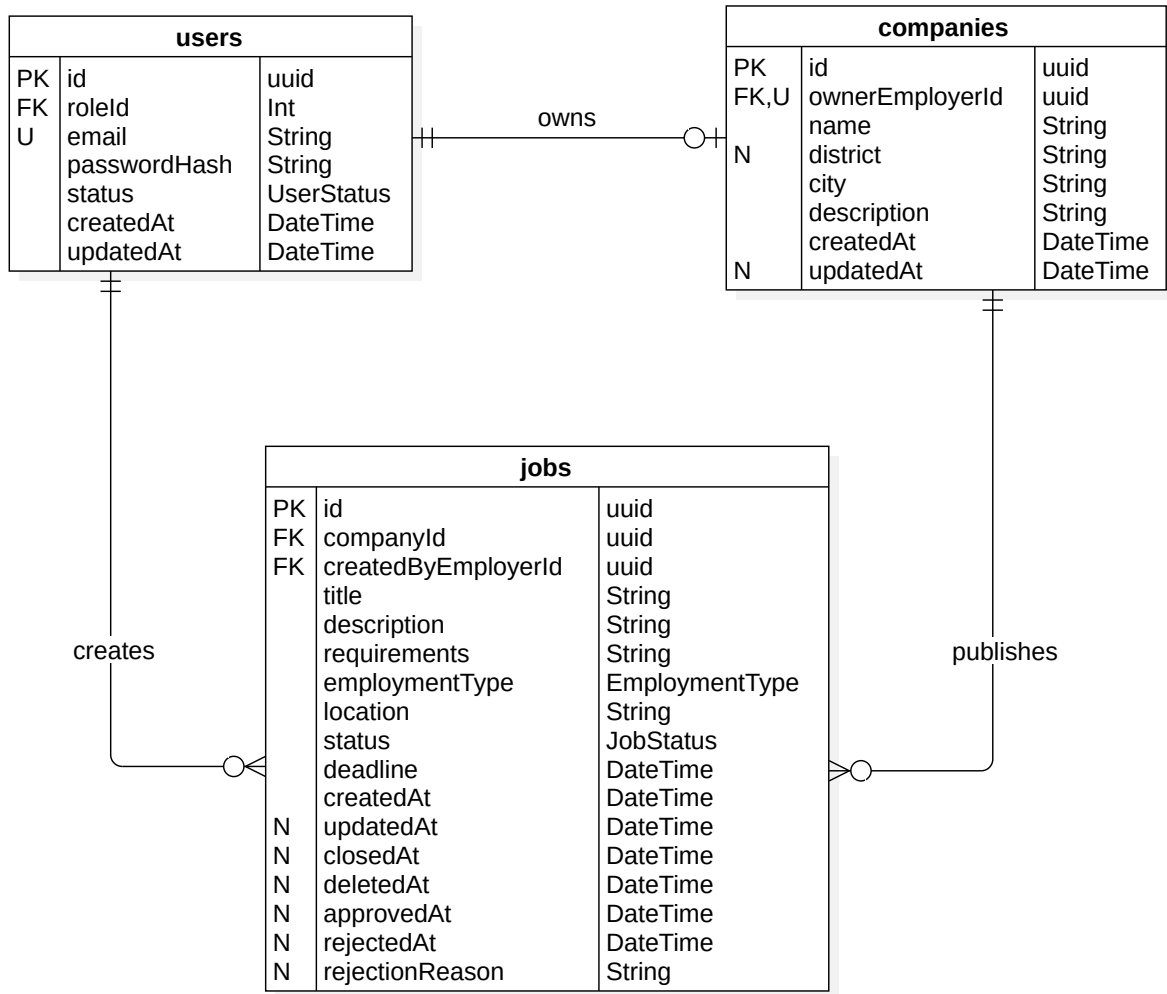


Figure 4.6: Company and Job Posting Entity Relationship Diagram.

An employer may own multiple companies, but each company belongs to exactly one employer. Similarly, an employer may create multiple job postings, but each job posting is created by exactly one employer. Each job posting belongs to exactly one company, while a company may publish any number of job postings. The status attribute of a job posting tracks its lifecycle — **PENDING_APPROVAL**, **ACTIVE**, **REJECTED**, **CLOSED**, **EXPIRED**, and **DELETED** — which is the backbone of the Employer Job Management module.

4.3.3 Job Application

Figure 4.7 details the entities involved in the job application workflow.

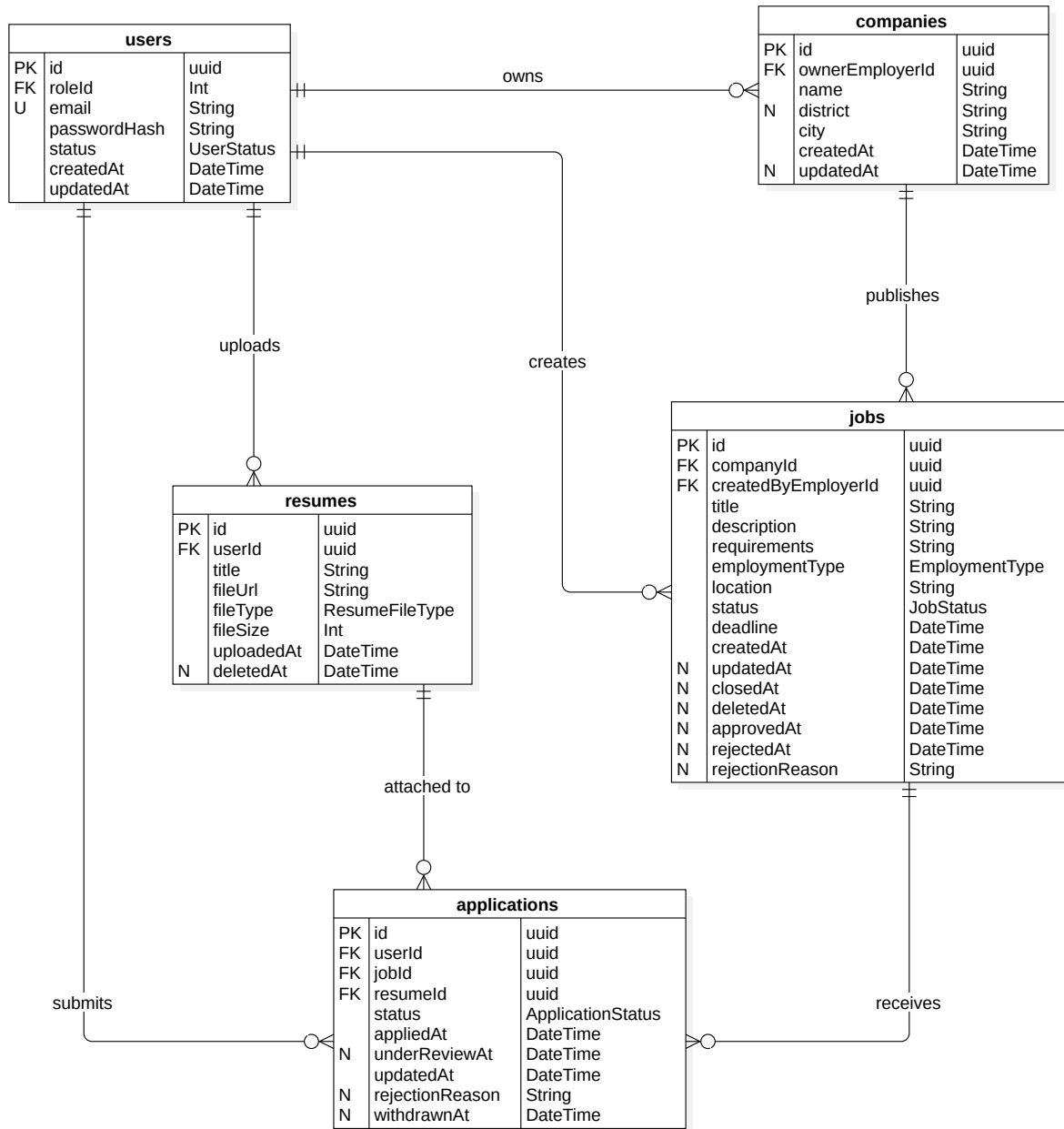


Figure 4.7: Job Application Entity Relationship Diagram.

A job seeker may upload multiple resumes and submit multiple applications, but each resume and each application belongs to exactly one job seeker. Each application is submitted for exactly one job posting, while a job posting may receive any number of applications. A resume may be attached to multiple applications, but each application references exactly one resume. The application's status attribute — **SUBMITTED**, **UNDER_REVIEW**, **ACCEPTED**, **REJECTED**, and **WITHDRAWN** — records where the application stands in the review process, and a unique constraint on the user-job pair prevents a candidate from applying to the same posting twice.

4.4 Backend Design

This section details the internal design of the Backend API container introduced in Section 4.2. It first describes the project structure and shared infrastructure common to

all modules, then presents the two featured business modules following the same template: an introduction, a class diagram, and a design explanation. The class diagrams of the remaining modules are presented in the appendix.

4.4.1 Backend Project Structure

The backend source code is organized as follows:

```
src
├── app.ts
├── server.ts
├── authentication.ts
├── api-external
├── api-internal
├── api-shared
├── email
├── generated
├── lib
├── security
└── utils
```

Table 4.2 summarizes the purpose of each top-level entry.

Table 4.2: Backend Project Structure.

Entry	Purpose
app.ts, server.ts	Wire the Express application: register global middleware and the generated routes, and bootstrap the HTTP server.
authentication.ts	Implements <code>expressAuthentication</code> for tsoa: verifies the JWT on protected routes and rejects banned accounts.
api-external	Implements business modules that expose application functionality to external users through RESTful APIs.
api-internal	Implements internal modules that support administrative and platform management operations.
api-shared	Provides reusable guards and logic shared across multiple business domains to reduce duplication.
email	Wraps the mail transport and holds transactional email templates.
generated	Contains files generated by tsoa during the build and should not be modified manually.
lib	Maintains shared library instances and application-wide configurations used by multiple modules.
security	Defines the <code>CurrentUser</code> type that carries the authenticated user's identifier and role into services.
utils	Provides general-purpose utilities such as <code>HttpError</code> , <code>PasswordHasher</code> , <code>JwtService</code> , <code>ResetTokenUtils</code> , and <code>FileStorageService</code> .

4.4.2 Shared Backend Components

A small set of reusable infrastructure components is shared across all business modules, avoiding duplication and ensuring consistent behavior.

- **JWT Authentication:** issues and verifies signed JSON Web Tokens carrying the user's identifier and role, and is enforced through the `expressAuthentication` hook on protected routes.
- **Role Guards:** helpers such as `assertJobSeeker` and `assertEmployer` reject requests from users who lack the required role, applied as the first step of every service method.
- **HttpError:** a uniform error type carrying an HTTP status and message, translated into the proper JSON error response.
- **Prisma Client:** the generated ORM client, instantiated once in `lib/prisma.ts` and imported by services for all database access.
- **File Storage:** abstracts file persistence behind a common interface, used for avatar and resume uploads.
- **Email Service:** wraps NodeMailer to send transactional emails, such as password reset notifications.
- **PasswordHasher and ResetTokenUtils:** wrap BCrypt and token generation for credential handling and password recovery.

4.4.3 Job Application Module

Introduction

The Job Application Module lets job seekers apply to active job postings using one of their resumes, view their own submitted applications, and withdraw an application while it is still eligible. It resides in `api-external`, and every endpoint requires both a valid JWT and the `JOB_SEEKER` role.

Class Diagram

Figure 4.8 presents the class diagram of the Job Application Module.

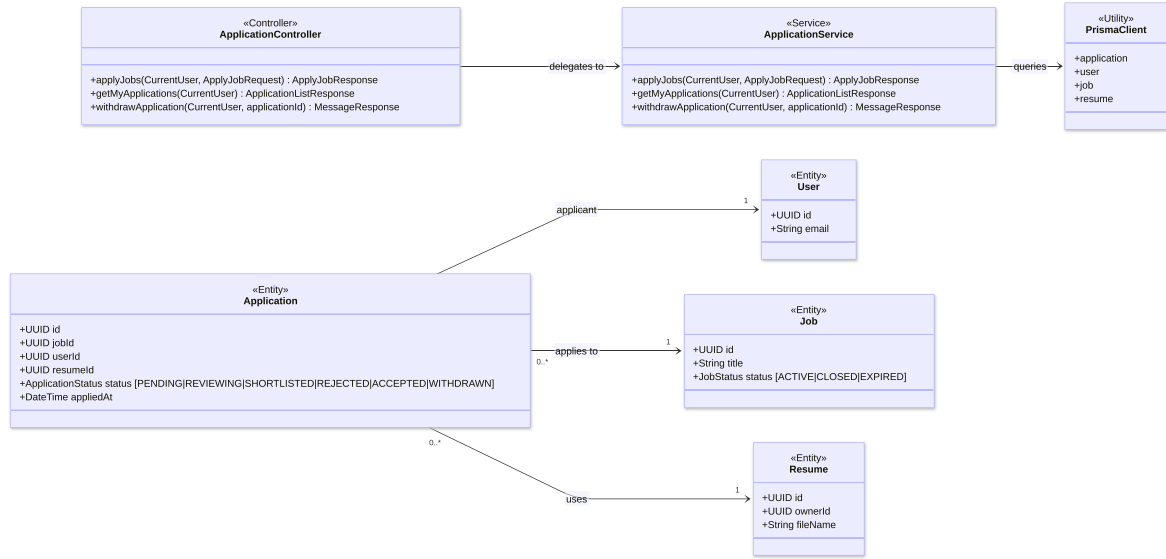


Figure 4.8: Job Application Module Class Diagram.

Design Explanation

Following the thin controller, rich service principle, ApplicationController exposes three REST endpoints and delegates each directly to a matching method on ApplicationService, without containing business logic itself. ApplicationService depends on the shared PrismaClient, which queries the User, Job, Resume, and Application entities whose relationships are detailed in Figure 4.7.

Every ApplicationService method begins with the same guard: the caller must hold the `JOB_SEEKER` role, otherwise the request is rejected. Beyond this shared guard, each method implements one use case:

- `applyJob` verifies that the referenced job exists and is **ACTIVE**, that the referenced resume exists and belongs to the caller, and that no prior application exists for the same job and user pair, before creating the application with an initial status of **SUBMITTED**.
- `getMyApplications` returns all applications submitted by the caller, joining through the related job and company to return a flattened summary alongside each application's status and submission date.
- `withdrawApplication` verifies that the application belongs to the caller and is still in **SUBMITTED** status, rejecting the request once it has moved further along the review process, then updates its status to **WITHDRAWN** and records the withdrawal timestamp.

4.4.4 Employer Job Management Module

Introduction

The Employer Job Management Module covers the capabilities exposed to employers: managing their company profile, managing the lifecycle of their job postings, and reviewing submitted applications. Together with the Job Application Module, it forms the complete round trip of a recruitment event studied in this thesis.

Class Diagram

Figures 4.9, 4.10, and 4.11 present the class diagrams of the Employer Job Management Module.

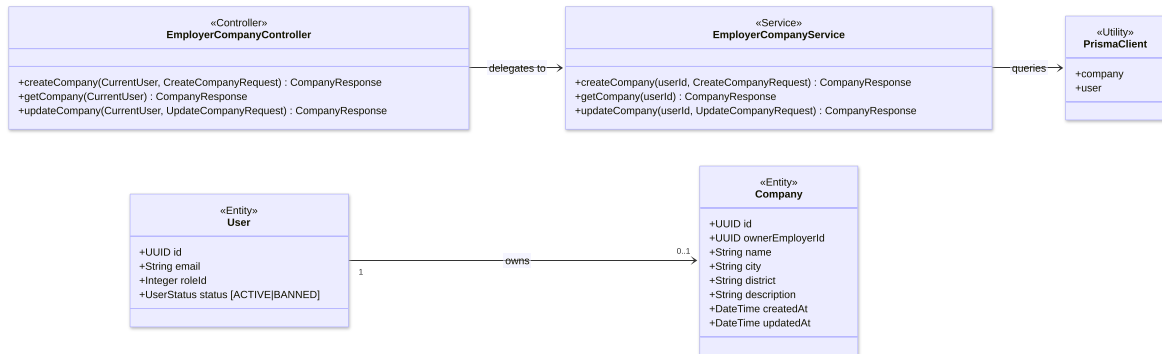


Figure 4.9: Company Profile Management Class Diagram.

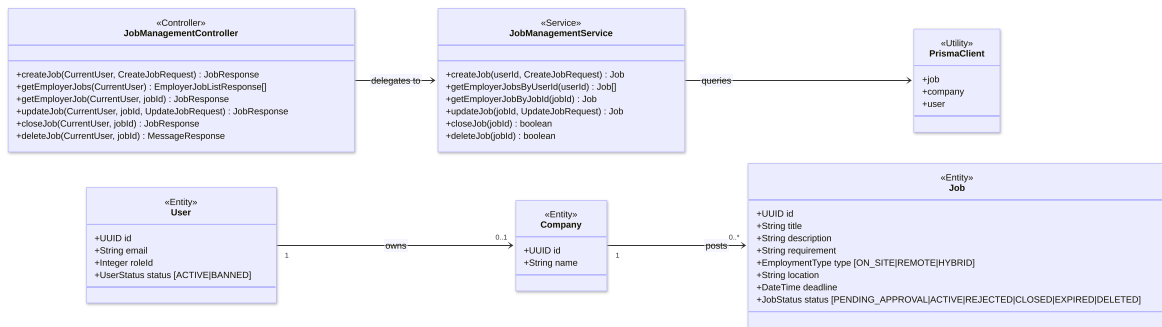


Figure 4.10: Job Posting Management Class Diagram.

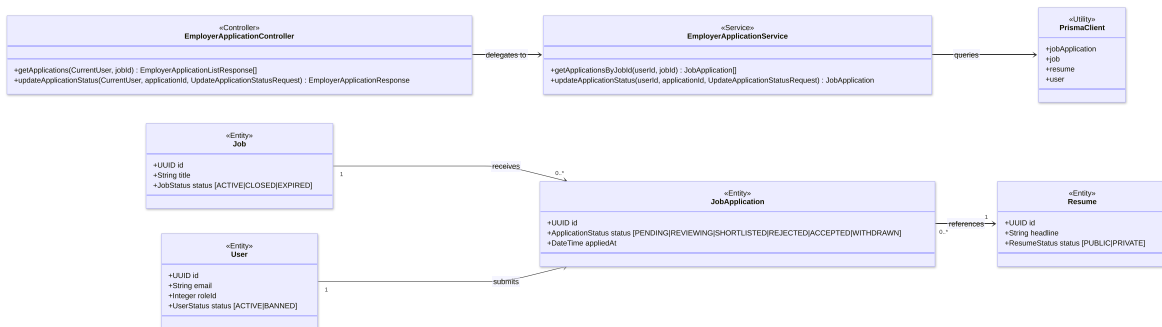


Figure 4.11: Employer Application Management Class Diagram.

Design Explanation

Across all three diagrams, each Controller delegates directly to a matching Service method, with no business logic in the controller itself, and each Service depends on the shared PrismaClient and the `assertEmployer` role guard.

Company Profile Management: `EmployerCompanyController` exposes `createCompany`, `getCompany`, and `updateCompany`, delegated to `EmployerCompanyService`. An employer owns at most one company, so none of these operations take a company identifier; the owning employer is instead resolved from the authenticated user.

Job Posting Management: `JobManagementController` exposes the full lifecycle of a job posting: `createJob`, `getEmployerJobs`, `getEmployerJob`, `updateJob`, `closeJob`, `reopenJob`, and `deleteJob`, delegated to `JobManagementService`. The Job status attribute includes `PENDING_APPROVAL` and `REJECTED`, reflecting that newly created and edited postings are subject to moderation before becoming publicly visible.

Application Review: `EmployerApplicationController` exposes `getJobApplications`, `getJobApplicationDetail`, `putApplicationUnderReview`, and `updateApplicationStatus`, delegated to `EmployerApplicationService`. This complements the `SUBMITTED` and `WITHDRAWN` transitions initiated by the job seeker: here, the employer moves an application through the remaining `UNDER_REVIEW`, `ACCEPTED`, and `REJECTED` statuses.

4.5 Frontend Design

The frontend is a React single-page application. Pages are grouped in feature folders that mirror the backend module structure, so a feature can be traced end to end: `ui-external/applications/` mirrors `api-external/job-application/`, and `ui-external/employer-job-management/` mirrors `api-external/employer-job-management/`. Reusable UI is separated into `ui-shared/`. Routing is role-based: `RequireAuth` guards wrap route groups so that a Job Seeker-only area and an Employer-only area are rendered only for the matching role. All API calls go through a client generated from the backend's OpenAPI specification, so the request and response types used by the pages always match the backend contract.

4.5.1 Job Application UI

The Job Application UI consists of two pages. The apply page, `ApplyJobPage` at `/jobs/:jobId/apply`, loads the chosen posting and the user's resumes, presents the job summary together with a `ResumeSelector`, and submits the selected resume. The submit errors are handled by HTTP status: a 409 reports that the user already applied, a 401 prompts re-login, and a 403 reports that only job seekers can apply. The tracking page, `MyApplicationsPage` at `/applications`, lists the user's applications through `ApplicationCard` components that show the posting, company, status badge, and submission date. Withdrawing an application is confirmed through a shared `ConfirmationDialog`; on success the card status is updated locally to `WITHDRAWN`.

4.5.2 Employer Job Management UI

The Employer Job Management UI covers two pages. `PostJobPage` at `/employer/jobs/new` collects a posting through `JobPostingForm`, which validates each field and converts the chosen calendar date into an ISO deadline at the end of that day via the shared deadline helper. `MyJobPostingsPage` at `/employer/jobs` lists the employer's postings with a `JobStatusBadge`; close, re-open, and delete actions each require confirmation in a

`ConfirmationDialog` and the list is refreshed after the mutation. The company profile page, `MyCompanyPage`, manages the employer's company. The application review screens that use the employer application review endpoints are still in progress at the time of writing.

4.6 Summary

This chapter presented the technical design of the platform. The system follows a layered, feature-oriented architecture split into a React frontend, a Node.js/Express API, and a PostgreSQL database. The database design centered on the entity groups that serve the two featured modules: company and job posting, and job application. The backend design introduced the project structure, the shared infrastructure components, and the class-level design of the Job Application and Employer Job Management modules. The frontend design described the page structure for both featured modules. The next chapter describes how this design was realized in code.

Chapter 5

Implementation

5.1 Overview

The design in Chapter 4 is implemented as a pnpm monorepo with two applications. The backend, `apps/api`, is a Node.js/Express service written in TypeScript: it uses Prisma for database access and `tsoa` to generate both the OpenAPI specification and the route handlers from annotated controllers. The frontend, `apps/web`, is a React single-page application built with Vite and TypeScript. It uses the generated OpenAPI client for all API calls, Tailwind CSS with `shadcn/ui` components for styling, and React Router for navigation.

The two applications exchange data exclusively over REST: the API exposes JSON endpoints and the web client calls them through generated functions whose request and response types match the backend DTOs. Because the client is regenerated from the backend's OpenAPI specification, a change to a response type on the server is reflected in the client at the next build instead of drifting silently.

5.1.1 Request Pipeline

Every request follows the same path through the backend:

HTTP request

- > `generated/routes.ts` (tsoa-generated route table)
- > `Controller` (tsoa decorators; routing + status codes)
- > `Service` (business rules; owns all Prisma access)
- > `Prisma Client` (SQL)
- > `PostgreSQL`

`tsoa` reads the `@Route`, `@Get`/`@Post`/`@Patch`, and `@Security` decorators on each controller and emits `routes.ts` plus `swagger.json`. The `@Security("jwt", [role])` decorator marks an endpoint as protected and declares the required scope; at runtime, `tsoa` calls `expressAuthentication`, implemented in `authentication.ts`, which verifies the JWT, rejects banned accounts, and attaches the authenticated user as `currentUser` on the request. The service then re-checks the role itself as defense in depth.

5.1.2 Error Handling

Services raise a uniform `HttpError(status, message)`; a global error handler converts it into a JSON response, while validation performed by `tsoa` on the incoming body is answered automatically. The status codes used throughout the API are summarized in Table 5.1.

Table 5.1: HTTP Status Code Conventions.

Code	Meaning
400	Validation failure, unmet precondition, or illegal status transition.
401	Missing or invalid access token.
403	Wrong role, banned account, or request on a resource the caller does not own.
404	Resource does not exist or must not be revealed to the caller.
409	The resource is already in the requested state (for example, a duplicate application).

5.2 Backend Implementation

5.2.1 Job Application Module

The module lives in `api-external/job-application/` and exposes the endpoints in Table 5.2, all protected with the `JOB_SEEKER` scope.

Table 5.2: Job Application Endpoints.

Endpoint	Purpose
POST <code>/applications</code>	Submit an application for a job using a chosen resume.
GET <code>/applications/my</code>	List the caller's applications with job and company summary.
PATCH <code>/applications/{id}/withdraw</code>	Withdraw a submitted application.

applyJob The core of the module is the `applyJob` method, which guards every business rule from the use case specification before any row is written. The checks run in order: the caller must be a job seeker; the referenced job must exist and be **ACTIVE**, otherwise the posting is unreachable; the chosen resume must exist, be not deleted, and belong to the caller; and no application may already exist for the same user and job pair. Only when all checks pass is an `Application` row created with the initial status **SUBMITTED**. The following excerpt shows the early guard sequence.

```
assertJobSeeker(currentUser);

const job = await prisma.job.findUnique({
  where: { id: requestBody.jobId },
  select: { id: true, status: true },
});
if (!job) throw new HttpError(404, "Job not found.");
```

```

if (job.status !== "ACTIVE") throw new HttpError(400,
  "Job posting is no longer available.");

const resume = await prisma.resume.findFirst({
  where: { id: requestBody.resumeId, deletedAt: null },
});
if (!resume) throw new HttpError(404, "Resume not found.");
if (resume.userId !== currentUser.id)
  throw new HttpError(403, "You do not own this resume.");

const application = await prisma.application.findUnique({
  where: { userId_jobId: { userId: currentUser.id, jobId: job.id } },
});
if (application) throw new HttpError(409,
  "You have already applied for this job.");

```

The duplicate check relies on the composite unique constraint `userId_jobId` defined on the `Application` table, which makes the one-application-per-job rule enforceable at the database level as well as in the service.

getMyApplications This method returns only the caller’s applications. Because an application relates to a job, and a job relates to its company, a single Prisma query joins those relations and maps each row to a flattened list item carrying the application status and submission date together with the job and company summary – the fields the tracking page displays.

withdrawApplication Withdrawal verifies ownership and then checks that the application is still in `SUBMITTED` status. An application that has moved into review, been decided, or already been withdrawn cannot be withdrawn again. On success the status is set to `WITHDRAWN` and the `withdrawnAt` timestamp is recorded; the row itself is kept for the job seeker’s history.

5.2.2 Employer Job Management Module

This module spans three folders in `api-external`: `employer-company/`, `employer-job-management/` and `employer-application-management/`, all protected with the `EMPLOYER` scope.

Company Profile

The company controller is mounted at `/employer/company` and provides `POST` (create), `GET` (view my company), and `PATCH` (update). Because an employer owns at most one company, no company identifier is ever sent by the client: the service looks the company up by `ownerEmployerId`, resolved from `currentUser`. The owner is therefore taken from the authenticated token, never from the request body.

Job Posting Lifecycle

The endpoints in Table 5.3 implement the full lifecycle of a job posting. Every mutating method first loads the posting through a shared helper that both checks existence and

confirms the caller created it, answering 404 when the posting does not exist and 403 when the caller is not its owner.

Table 5.3: Job Posting Endpoints.

Endpoint	Purpose
GET /employer/jobs	List the employer's postings, newest first.
GET /employer/jobs/{id}	Detail of one owned posting.
POST /employer/jobs	Create a posting (status <code>PENDING_APPROVAL</code>).
PATCH /employer/jobs/{id}	Edit a posting; resets it to <code>PENDING_APPROVAL</code> .
PATCH /employer/jobs/{id}/close	Close an <code>ACTIVE</code> posting.
PATCH /employer/jobs/{id}/reopen	Re-open a <code>CLOSED</code> posting.
PATCH /employer/jobs/{id}/delete	Soft-delete a posting.

Input validation Before creating or editing a posting, the service trims and validates every required field: the title and location are capped at 255 characters and the description and requirements at 10000. The deadline is parsed as a date and must lie in the future. A company profile must already exist, because the posting's `companyId` is resolved from the employer's own company, not provided by the client.

Status transitions Each lifecycle action is a guarded state transition. Only an `ACTIVE` posting can be closed; only a `CLOSED` posting can be re-opened, and only if its deadline is still in the future; editing is allowed for every status except `DELETED` and `EXPIRED`. Deleting is a soft delete: the status is set to `DELETED` and a `deletedAt` timestamp is recorded, so the record and its applications are preserved for history. Whenever a posting is edited or re-opened, its status returns to `PENDING_APPROVAL` and the moderation timestamps (`approvedAt`, `rejectedAt`, `rejectionReason`) are cleared, so a previously rejected posting does not appear to be rejected again.

Employer Application Review

The endpoints in Table 5.4 close the loop on the application flow: the employer sees what the job seeker submitted and moves it towards a decision.

Table 5.4: Employer Application Review Endpoints.

Endpoint	Purpose
GET /employer/jobs/{id}/applications	List applications received by a posting.
GET /employer/applications/{id}	Full detail of one application (candidate and resume).
PATCH /employer/applications/{id}/under-review	Move SUBMITTED to UNDER_REVIEW .
PATCH /employer/applications/{id}/status	Accept or reject an application under review.

The application list and the detail both verify that the application belongs to a job posting owned by the employer before returning data. The two state-moving methods apply the same status gates: an already **WITHDRAWN**, **ACCEPTED**, or **REJECTED** application cannot be changed again, under-review requires the current status **SUBMITTED**, and accept/reject requires the current status **UNDER_REVIEW**. On rejection an optional reason is trimmed, capped at 1000 characters, and stored in `rejectionReason`; the decision time is recorded in `decidedAt`.

5.2.3 Remaining Modules

The remaining modules follow the same controller–service pattern, but are covered here briefly.

Authentication `AuthService` implements registration, login, logout, password reset requests, and password reset. Passwords are hashed with BCrypt before storage and compared at login; a banned account is rejected at login and by the JWT middleware on every protected request. Password reset tokens are hashed before being stored, carry an expiration, and are single-use; the reset email is sent through the email service.

Profile `ProfileService` lets a user read their profile and job preferences, update them with partial-update semantics (an omitted field is left unchanged, an explicit `null` clears it), change the avatar with file type and size checks, and change the password after verifying the current one.

Resume `ResumeService` lists, uploads, and deletes resumes. Uploads accept only PDF, DOC, and DOCX files up to 5 MB, stored through the file storage abstraction. A resume referenced by an existing application is soft-deleted so old applications keep a valid reference; otherwise it is removed entirely.

Job Discovery `JobDiscoveryService` powers public job search and job detail. Search restricts results to **ACTIVE** postings and matches an optional keyword against the job title or company name and an optional location against the job location or company city and district; the public company profile endpoints expose company details and their active postings.

Back Office Moderation The `api-internal/job-moderation/` module defines the approval, rejection, and deletion operations for moderators. At the time of writing its frontend is not yet implemented and the approval operation is still being wired, so job postings reach `ACTIVE` through moderation only partially; this is noted as in-progress work rather than covered as a finished feature.

5.3 Frontend Implementation

This section describes how the two featured modules are realized in the React frontend. The Employer Application Review use case does not have a user interface yet – it is implemented at the API level as described above, and its UI will be written later.

Pages and components live in feature folders that mirror the backend, and each route group is wrapped with `RequireAuth` so that Job Seeker-only and Employer-only areas render only for the matching role. The API client is generated from the backend's OpenAPI spec, so calls such as `getMyApplications(...)` or `applyJobs(...)` are typed end to end, and the shared `apiClient` attaches the stored access token and redirects to login on a 401.

5.3.1 Job Application UI

Apply page `ApplyJobPage (/jobs/:jobId/apply)` loads the posting and the user's resumes on mount. The page shows the job summary and a `ResumeSelector`; the first available resume is preselected. On submit it calls the `applyJobs` API and maps errors by status: a 409 explains that the user already applied, a 401 that the session expired, and a 403 that the role is not allowed. A success state replaces the form with a confirmation and links to the applications page.

Tracking page `MyApplicationsPage (/applications)` fetches the user's applications and renders one `ApplicationCard` per row, showing the posting, company, status badge, and application date. Withdrawal runs through a shared `ConfirmationDialog`; after the API succeeds, the card's status is updated locally to `WITHDRAWN` without refetching the whole list.

5.3.2 Employer Job Management UI

Post a job `PostJobPage (/employer/jobs/new)` collects the posting through `JobPostingForm`: title, description, requirements, location, a segmented employment-type control, and a deadline date picker. The deadline picker only allows dates from tomorrow onward, and on submit the helper `toDeadlineIso` converts the chosen calendar date into an ISO timestamp at the end of that day in the employer's local time zone, matching the backend rule that a deadline must be strictly in the future. After submission the page returns to the list, where the new posting is visible in `PENDING_APPROVAL`.

My job postings `MyJobPostingsPage (/employer/jobs)` lists the employer's postings from newest to oldest through `JobPostingCard`, which shows the summary fields and a `JobStatusBadge`. The close, re-open, and delete actions each require confirmation in a `ConfirmationDialog` with copy that reflects the real consequence of the action; on success the list is refreshed so the card reflects the new status.

5.3.3 Remaining Pages

The remaining pages follow the same conventions: the auth pages (login, register, forgot/reset password), the landing and job discovery pages (job search, job detail, company profile), the profile page for personal information and resume upload, and the employer company page. Each keeps to the reach – call the generated API client – render the result – handle the error pattern used in the featured modules.

5.4 Summary

The platform was implemented as a two-application monorepo. The backend follows the controller–service architecture established in the design chapter: `tsoa` provides routing and the OpenAPI contract, services own all business rules, and `Prisma` provides typed database access. The two featured modules were implemented in full – the job seeker can apply, track, and withdraw applications, while the employer can manage a company, run the lifecycle of a job posting, and review its applications – and the remaining modules follow the same pattern with the moderation work still in progress. The frontend mirrors the backend with role-guarded pages and a generated, typed API client.

Chapter 6

Testing

Chapter 7

Conclusion

Bibliography